

PR Audit Report

Repository	dipak-shaaki/demo-git-test-2
PR Number	#2
Risk Level	HIGH

PR Audit Report

Summary

This PR adds new user-related API endpoints and a service layer for fetching a single user and listing users from a SQLite database.

What the Developer Was Trying to Do

The developer appears to be implementing basic user management/profile APIs:

- GET /{user_id} to fetch a user by ID
- GET / to list users, optionally filtered by role

The implementation is backed by a new SQLite service module.

Security Issues (Bandit + Dependabot + SonarCloud)

1) SQL injection risk in `get\_user\_by\_id`

Full function:

```
def get_user_by_id(user_id: str):  
    conn = sqlite3.connect(DB_PATH)
```

```

cursor = conn.cursor()

query = f"SELECT * FROM users WHERE id = '{user_id}'" # <- ISSUE HERE
result = cursor.execute(query).fetchall()

conn.close()
return result

```

Why this is problematic / risk:

- The query is built using string interpolation with untrusted input (`user_id`).
- This can allow SQL injection if an attacker supplies a crafted `user_id`.
- Risk includes unauthorized data access, data corruption, or database compromise depending on the SQLite permissions and surrounding application behavior.

Corrected version:

```

def get_user_by_id(user_id: str):
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()

    query = "SELECT * FROM users WHERE id = ?"
    result = cursor.execute(query, (user_id,)).fetchall()

    conn.close()
    return result

```

Official documentation:

- Bandit B608: https://bandit.readthedocs.io/en/1.9.4/plugins/b608_hardcoded_sql_expressions.html
- Python `sqlite3` parameter substitution:
<https://docs.python.org/3/library/sqlite3.html#sqlite3.Cursor.execute>

2) SQL injection risk in `list_users`

Full function:

```

def list_users(role: str):
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()

    if role == "admin":

```

```

query = "SELECT * FROM users"
else:
    query = f"SELECT * FROM users WHERE role = '{role}'" # <- ISSUE HERE

result = cursor.execute(query).fetchall()

conn.close()
return result

```

Why this is problematic / risk:

- The role value is directly interpolated into the SQL statement.
- Even though there is a special-case branch for "admin", all other values are still vulnerable to SQL injection.
- This can expose or alter user data if an attacker controls the role parameter.

Corrected version:

```

def list_users(role: str):
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()

    if role == "admin":
        query = "SELECT * FROM users"
        result = cursor.execute(query).fetchall()
    else:
        query = "SELECT * FROM users WHERE role = ?"
        result = cursor.execute(query, (role,)).fetchall()

    conn.close()
    return result

```

Official documentation:

- Bandit B608: https://bandit.readthedocs.io/en/1.9.4/plugins/b608_hardcoded_sql_expressions.html
- Python sqlite3 parameter substitution:
<https://docs.python.org/3/library/sqlite3.html#sqlite3.Cursor.execute>

Code Quality Issues (Ruff + SonarCloud)

No Ruff or SonarCloud findings were reported for the changed files.

Teammate Comments Summary

No existing teammate review comments were provided.

Overall Risk Assessment

- **Risk Level:** HIGH
- **Reason:** The new database access code contains two SQL injection vectors in user-facing endpoints, which is a significant security issue.